
Établissement : EIDIA
Filière : Full Stack Multimédia

Rapport de Projet de Fin de Module

Module : Advanced Back end

Fleur & Atelier

*Plateforme e-commerce de création de bouquets personnalisés
Spring Boot 3 + React 18*

Réalisé par :

Nom & Prénom :

BENNANI GABSI Zineb

Encadré par :

Nom & Prénom :

OUANAN Mohammed

Année Académique

2025-2026

Résumé

Résumé — Ce rapport présente la conception et le développement de **Fleur & Atelier**, une plateforme e-commerce dédiée à la création de bouquets de fleurs personnalisés. Le projet repose sur une architecture full-stack combinant un backend **Spring Boot 3** structuré selon les principes de la *Clean Architecture*, et un frontend **React 18** interfacé via une API REST sécurisée par **JWT**.

La méthodologie adoptée suit une démarche itérative : analyse des besoins, modélisation UML, développement couche par couche et tests unitaires. Les principaux résultats comprennent un catalogue géré par l'administrateur, un *bouquet* comme entité métier persistante, un processus de commande transactionnel avec vérification du stock, un module de paiement extensible via le patron *Strategy*, et un suivi de livraison avec contrôle par rôle.

Abstract — This report presents the design and development of **Fleur & Atelier**, an e-commerce platform dedicated to the creation of personalised flower bouquets. The project is based on a full-stack architecture combining a **Spring Boot 3** backend structured according to Clean Architecture principles, and a **React 18** frontend interfaced through a REST API secured with JWT. Key achievements include a manageable article catalogue, a bouquet treated as a first-class business entity, a transactional ordering process with stock verification, an extensible payment module using the Strategy design pattern, and role-based delivery tracking.

Mots-clés : Spring Boot, React, Architecture Propre, JWT, e-commerce, Design Patterns, REST API, JPA/Hibernate, Sécurité.

Table des matières

Résumé	1
Liste des figures	4
Liste des tableaux	5
Introduction Générale	6
1 Problématique et Contexte	7
1.1 État de l’art et Revue de Littérature	7
1.1.1 Technologies logicielles utilisées	7
1.1.2 Outils et méthodologies	8
1.1.3 Travaux similaires	8
1.2 Problématique et Objectifs du Projet	8
1.2.1 Problématique identifiée	8
1.2.2 Objectifs spécifiques	9
2 Méthodologie	10
2.1 Analyse des Besoins	10
2.1.1 Acteurs du système	10
2.1.2 Cas d’utilisation principaux	10
2.2 Conception	10
2.2.1 Architecture en couches — Clean Architecture	10
2.2.2 Analyse des acteurs et cas d’utilisation	11
2.2.3 Modèle de domaine	12
2.2.4 Flux d’authentification JWT	13
2.3 Développement	13
2.3.1 Gestion du versionnement	14
2.3.2 Patrons de conception appliqués	14
3 Réalisation du Projet	15
3.1 Structure du Projet	15

3.2	Fonctionnalités Implémentées	15
3.2.1	Module Authentification	15
3.2.2	Module Bouquet — Entité Métier	16
3.2.3	Module Commande et Paiement Atomique	16
3.2.4	Patron Strategy — Module de Paiement	17
3.2.5	Règle du Snapshot	17
3.2.6	Verrouillage Optimiste	18
3.3	API REST — Endpoints Implémentés	19
3.4	Interface Utilisateur	19
	Conclusion et Perspectives	20

Table des figures

2.1	Architecture en couches du projet (Clean Architecture)	11
2.2	Diagramme de Cas d'Utilisation UML	12
2.3	Diagramme de Classes UML	12
2.4	Diagramme de séquence détaillé du processus d'authentification	13
3.1	Patron Strategy appliqué au module de paiement	17

Liste des tableaux

1.1	Composants Spring Boot utilisés	7
1.2	Outils de développement utilisés	8
2.1	Acteurs du système Fleur & Atelier	10
2.2	Patrons de conception appliqués	14
3.1	Principaux endpoints de l'API REST	19

Introduction Générale

Contexte

Le secteur de la vente de fleurs en ligne connaît une croissance significative, portée par la numérisation des commerces traditionnels et l'évolution des habitudes d'achat. Si de nombreuses boutiques ont développé une présence en ligne, rares sont celles qui proposent une expérience d'achat véritablement personnalisée permettant à l'utilisateur de *composer* son propre bouquet.

Fleur & Atelier s'inscrit dans cette problématique : le bouquet n'est pas un simple panier d'articles, mais une *entité métier* à part entière, composable, modifiable et persistante tout au long du parcours utilisateur.

Objectifs

Ce projet de fin de module vise à :

1. Concevoir une architecture logicielle **propre et modulaire** en appliquant les principes SOLID et la Clean Architecture.
2. Développer une **API REST** complète et sécurisée avec Spring Boot 3, couvrant catalogue, bouquet, commandes, paiement et livraison.
3. Implémenter une **authentification stateless** par JWT avec gestion des rôles (USER / ADMIN).
4. Créer une **interface React 18** réactive connectée via Axios.
5. Mettre en œuvre des **patrons de conception** : Strategy (paiement), Snapshot (lignes de commande) et verrouillage optimiste (stock).

Structure du rapport

- **Chapitre 1** : problématique, état de l'art, technologies.
- **Chapitre 2** : méthodologie, analyse des besoins, conception UML.
- **Chapitre 3** : réalisation, extraits de code, fonctionnalités.

Chapitre 1

Problématique et Contexte

1.1 État de l’art et Revue de Littérature

1.1.1 Technologies logicielles utilisées

Backend — Spring Boot 3

Spring Boot [1] est un framework Java open-source qui simplifie la création d’applications de production autonomes. La version 3 s’appuie sur Spring Framework 6 et requiert Java 17 minimum.

TABLE 1.1 – Composants Spring Boot utilisés

Composant	Rôle	Usage dans le projet
Spring Web MVC	API REST	Exposition des endpoints HTTP
Spring Data JPA	Persistance	Accès BD via Hibernate 6
Spring Security	Sécurité	Authentification JWT, RBAC
Spring Validation	Validation	Validation des DTO (Bean Validation)
Lombok	Productivité	Génération du code boilerplate
JWT	JWT	Génération et validation des tokens

Frontend — React 18 + Vite

React [2] est une bibliothèque JavaScript pour la construction d’interfaces utilisateur déclaratives. **Vite** [3] est un outil de build nouvelle génération offrant un serveur de développement ultra-rapide grâce aux modules ES natifs. Les bibliothèques complémentaires utilisées sont : Axios (client HTTP), Context API (état global), React Router DOM (navigation et routes protégées).

Base de données

Le projet utilise **H2** en mémoire pour le développement, permettant un démarrage rapide sans configuration externe. La couche de persistance est gérée par **JPA/Hibernate 6** [4] avec les annotations standard Jakarta.

1.1.2 Outils et méthodologies

TABLE 1.2 – Outils de développement utilisés

Outil	Catégorie	Usage
Git / GitHub	Versionnement	Suivi des modifications, branches
Maven	Build	Gestion des dépendances Java
IntelliJ IDEA	IDE	Développement Spring Boot
VS Code	IDE	Développement React
Postman	Tests API	Validation des endpoints REST
H2 Console	Base de données	Inspection des données en mémoire
PlantUML	Modélisation	Génération des diagrammes UML

1.1.3 Travaux similaires

- **Shopify / WooCommerce** : plateformes e-commerce généralistes sans personnalisation métier orientée composition de bouquets.
- **Spring PetClinic** [7] : exemple de référence Spring illustrant les bonnes pratiques de structuration JPA.
- **Clean Architecture (R. Martin)** [6] : ouvrage fondateur ayant guidé la structuration en couches du projet.

1.2 Problématique et Objectifs du Projet

1.2.1 Problématique identifiée

Les plateformes e-commerce existantes traitent le panier comme une collection d'articles sans signification métier propre. Pour un fleuriste, cette approche est insuffisante : la composition du bouquet est un acte créatif qui doit être persistée, modifiable, et soumis à des règles métier spécifiques (unicité d'un article par bouquet, vérification du stock, calcul du prix en temps réel). La gestion atomique du paiement et l'extensibilité du module de paiement sont des contraintes techniques supplémentaires.

1.2.2 Objectifs spécifiques

1. **Bouquet comme entité métier** : entité persistée appartenant à un utilisateur, composée de lignes (article + quantité).
2. **Processus de commande robuste** : paiement requis avant finalisation, stock décrémenté de manière atomique.
3. **Extensibilité du paiement** : ajout de nouveaux modes sans modifier le code existant (principe Ouvert/Fermé).
4. **Sécurité par rôle** : opérations d'administration réservées au rôle ADMIN.
5. **Snapshot des commandes** : immuabilité des données après paiement.

Chapitre 2

Méthodologie

2.1 Analyse des Besoins

2.1.1 Acteurs du système

TABLE 2.1 – Acteurs du système Fleur & Atelier

Acteur	Rôle	Périmètre d'action
Visiteur	Non authentifié	Consultation du catalogue, recherche
Client	USER (JWT)	Bouquet, commandes, paiement, livraison
Administrateur	ADMIN (JWT)	Gestion catalogue, commandes, livraisons
Système de paiement	Ext.	Traitement transactions (CB, PayPal)

2.1.2 Cas d'utilisation principaux

Les besoins fonctionnels ont été formalisés en 26 cas d'utilisation répartis en 7 modules : Authentification, Catalogue, Bouquet, Commandes, Paiement, Livraison et Administration.

2.2 Conception

2.2.1 Architecture en couches — Clean Architecture

L'architecture suit les principes de la *Clean Architecture* [6], avec des dépendances strictement orientées vers l'intérieur :

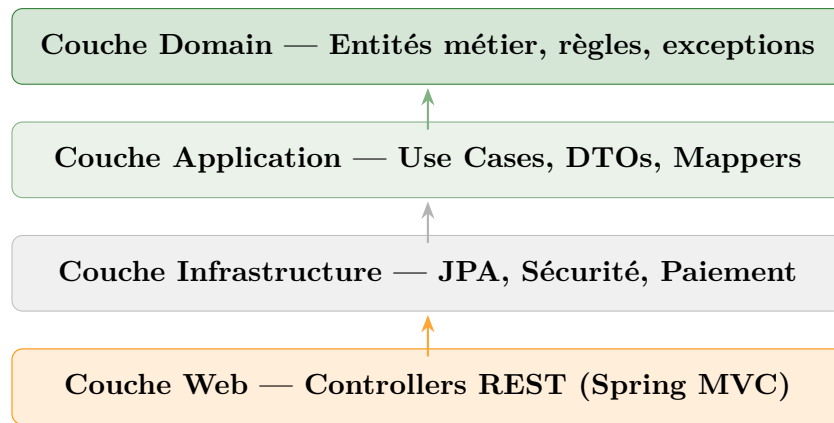


FIGURE 2.1 – Architecture en couches du projet (Clean Architecture)

2.2.2 Analyse des acteurs et cas d'utilisation

Analyse des acteurs Le système repose sur quatre acteurs principaux.

Le **Visiteur** représente un utilisateur non authentifié ayant accès aux fonctionnalités publiques telles que la consultation des articles et des catégories via les endpoints `GET /api/articles` et `GET /api/categories`.

Le **Client**, correspondant à un utilisateur authentifié portant le rôle

`USER` dans l'énumération `RoleUtilisateur`, accède aux fonctionnalités protégées telles que la gestion des bouquets, des commandes et le processus de paiement, sécurisé via JWT.

L'**Administrateur**, identifié par le rôle `ADMIN`, dispose de privilèges étendus lui permettant de gérer les articles, les livraisons et les uploads, protégés par des annotations de sécurité de type `@PreAuthorize("hasRole('ADMIN')")`.

Enfin, le **Système de paiement** constitue un acteur externe intégré via le patron de conception *Strategy*, représenté par l'interface `PaymentStrategy` et ses implémentations `CarteBancairePayment` et `PaypalPayment`, sollicité lors des opérations de paiement.

Le **Visiteur** est une généralisation du **Client** dans le modèle UML, ce dernier héritant des capacités de navigation et de consultation du premier tout en ajoutant des fonctionnalités transactionnelles.

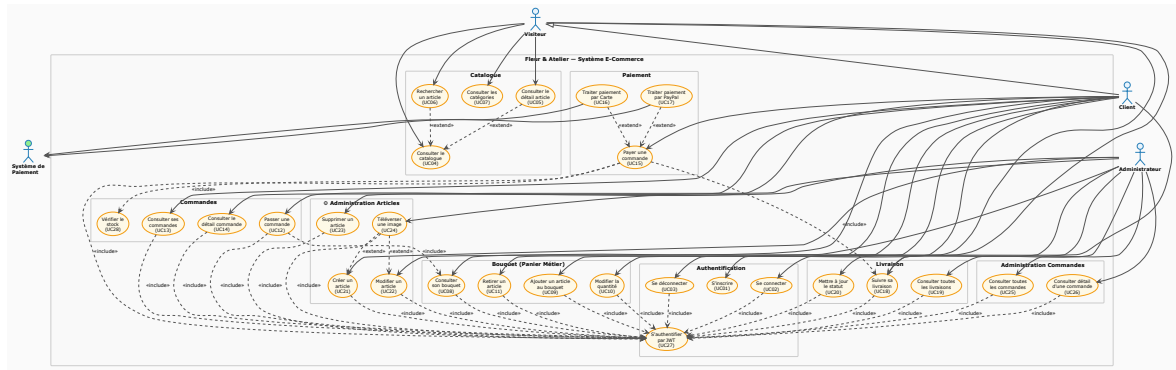


FIGURE 2.2 – Diagramme de Cas d'Utilisation UML

2.2.3 Modèle de domaine

Le modèle de domaine est composé de **8 entités JPA** et **3 énumérations**.

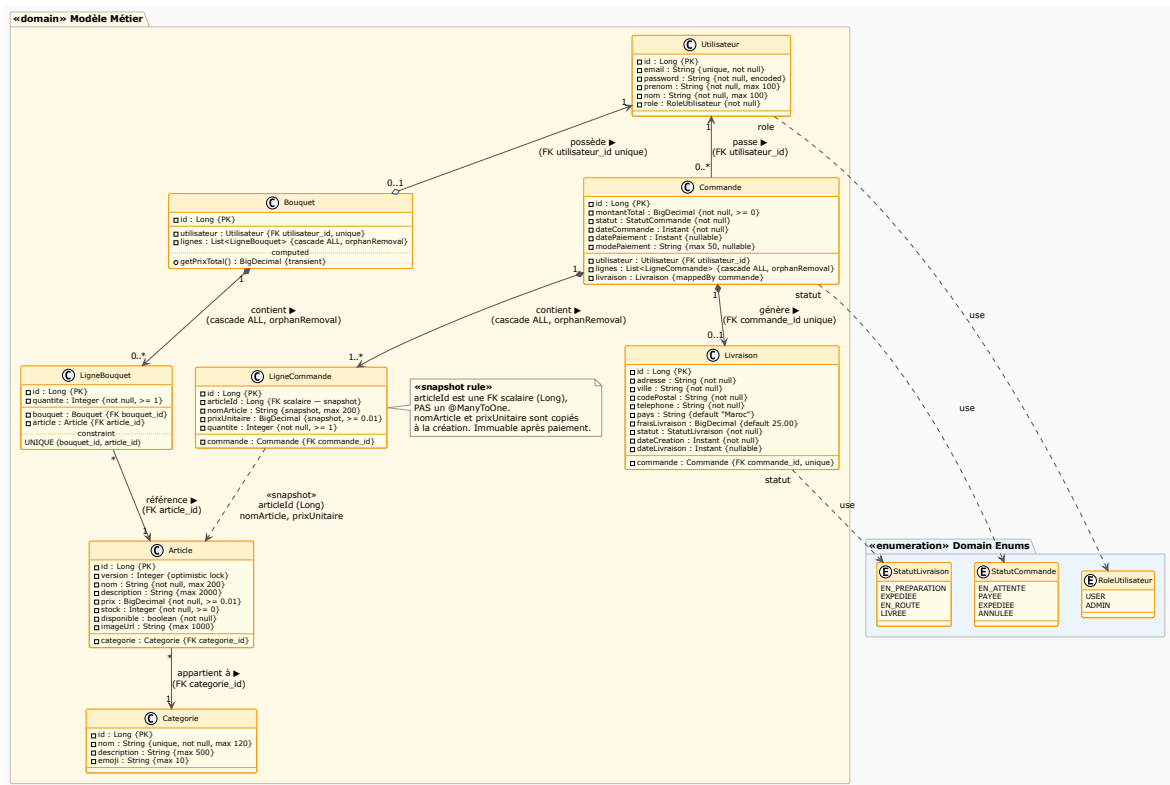


FIGURE 2.3 – Diagramme de Classes UML

2.3 Développement

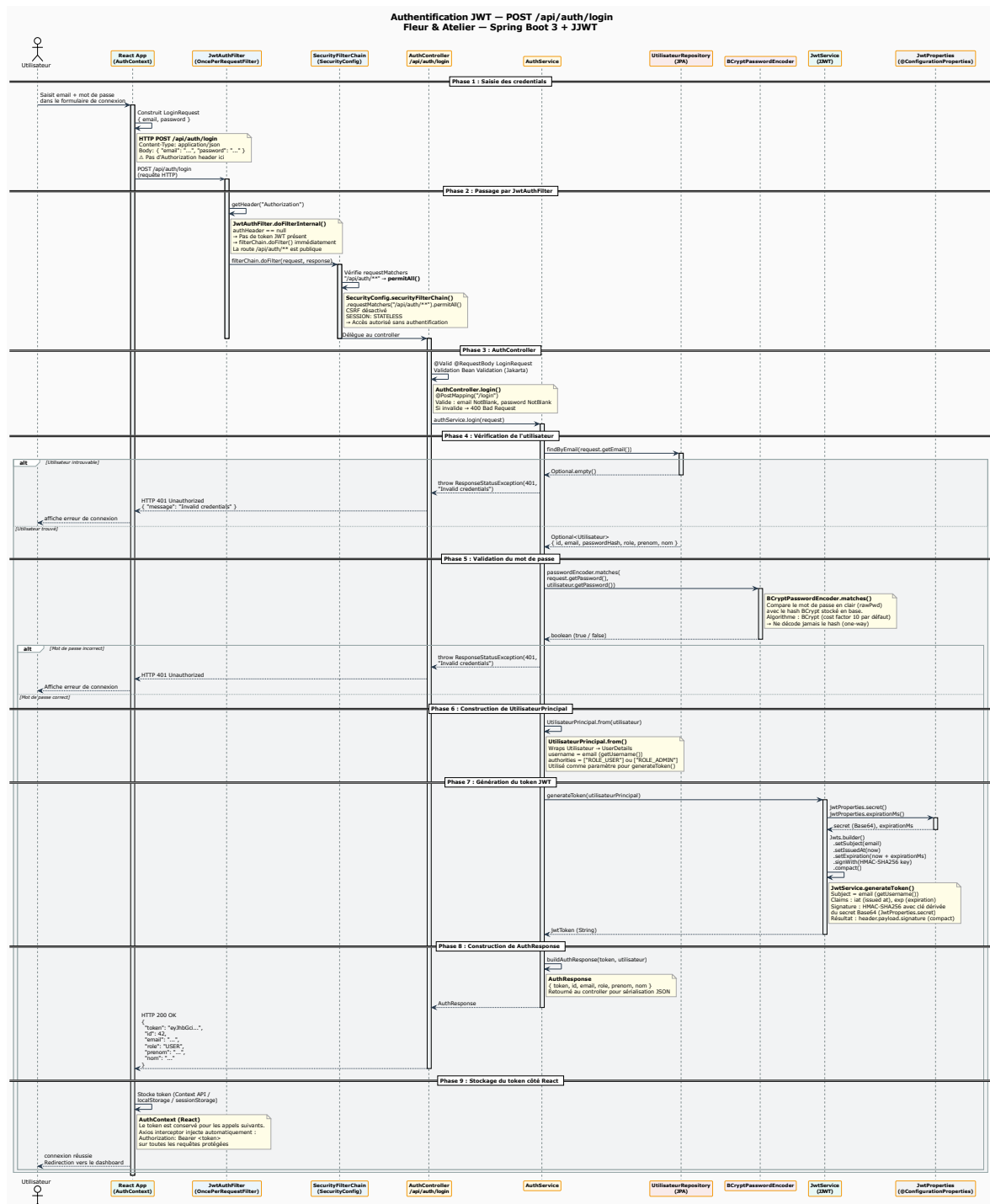


FIGURE 2.4 – Diagramme de séquence détaillé du processus d'authentification

2.3.1 Gestion du versionnement

Le projet a été géré avec **Git** et hébergé sur **GitHub**. La stratégie de branches adoptée :

- `main` : branche de production stable.
- `develop` : intégration des fonctionnalités.
- `feature/xxx` : branche par fonctionnalité (ex. `feature/payment-strategy`).

2.3.2 Patrons de conception appliqués

TABLE 2.2 – Patrons de conception appliqués

Patron	Localisation	Justification
Strategy	<code>PaymentContext</code>	Extensibilité sans modification du code existant
Repository	<code>*Repository</code>	Abstraction de l'accès aux données
DTO/Mapper	<code>application/dto</code>	Découplage couche web / domaine
Snapshot	<code>LigneCommande</code>	Immuabilité des données historiques
Builder	Toutes les entités	Construction fluide via Lombok

Chapitre 3

Réalisation du Projet

3.1 Structure du Projet

Le projet backend est organisé en quatre packages principaux :

```
com.fleuratelier.fleur_atelier_api/  
domain/ - pur métier, sans Spring  
model/ (8 entités JPA)  
enums/ (3 énumérations)  
exception/  
application/ - cas d'utilisation  
usecase/ (5 services)  
dto/ mapper/  
infrastructure/ - technique  
persistence/ security/ payment/  
web/ - controllers REST  
AuthController ArticleController  
BouquetController CommandeController  
PaielementController LivraisonController
```

3.2 Fonctionnalités Implémentées

3.2.1 Module Authentification

L'authentification repose sur Spring Security 6 avec le filtre `JwtAuthFilter` (`OncePerRequestFilter`) qui intercepte chaque requête. La configuration est entièrement *stateless* (`SessionCreationPolicy.STATELESS`).

```
1 public String generateToken(UserDetails userDetails) {  
2     Date now = new Date();  
3     Date expiry = new Date(now.getTime()  
4         + jwtProperties.expirationMs());
```



```

5      return Jwts.builder()
6          .setSubject(userDetails.getUsername()) // email
7          .setIssuedAt(now)
8          .setExpiration(expiry)
9          .signWith(getSigningKey()) // HMAC-SHA256
10         .compact();
11     }

```

Listing 3.1 – Génération du token JWT

Les rôles sont encodés dans `UtilisateurPrincipal` : `"ROLE_" + role.name() → ROLE_USER / ROLE_ADMIN`. Les routes admin sont protégées par `@PreAuthorize("hasRole('ADMIN')")`.

3.2.2 Module Bouquet — Entité Métier

L'innovation principale est le traitement du bouquet comme entité persistée. La méthode `getOrCreateBouquet()` crée automatiquement un bouquet si l'utilisateur n'en possède pas encore :

```

1 private Bouquet getOrCreateBouquet(Utilisateur utilisateur) {
2     return bouquetRepository
3         .findByUtilisateurId(utilisateur.getId())
4         .orElseGet(() -> bouquetRepository.save(
5             Bouquet.builder()
6                 .utilisateur(utilisateur)
7                 .build()));
8 }

```

Listing 3.2 – Création automatique du bouquet

La contrainte `UNIQUE(bouquet_id, article_id)` garantit qu'un même article n'apparaît qu'une seule fois par bouquet.

3.2.3 Module Commande et Paiement Atomique

La méthode `payerCommande()` s'exécute dans une seule transaction `@Transactional` :

1. Vérification du stock pour chaque article de la commande.
2. Traitement du paiement via le patron *Strategy*.
3. Décrémentation atomique du stock.
4. Mise à jour du statut de la commande vers `PAYEE`.
5. Création automatique d'une Livraison.

```

1 @Transactional
2 public CommandeResponse payerCommande(PaiementRequest req,
3                                         Long utilisateurId) {
4     // 1. Verification stock

```

```

5   for (LigneCommande ligne : commande.getLignes()) {
6       Article a = articleRepository
7           .findById(ligne.getArticleId())
8           .orElseThrow(...);
9       if (a.getStock() < ligne.getQuantite())
10          throw new InsufficientStockException(...);
11   }
12   // 2. Paiement via Strategy
13   PaymentResult result =
14       paymentContext.pay(req.getModePaiement(), commande);
15   if (!result.isSuccess())
16       throw new ResponseStatusException(BAD_REQUEST, ...);
17   // 3-5. Stock + statut PAYEE + creation Livraison ...
18 }

```

Listing 3.3 – Coeur du traitement de paiement atomique

3.2.4 Patron Strategy — Module de Paiement

Le patron *Strategy* permet d'ajouter un nouveau mode de paiement sans modifier le code existant :

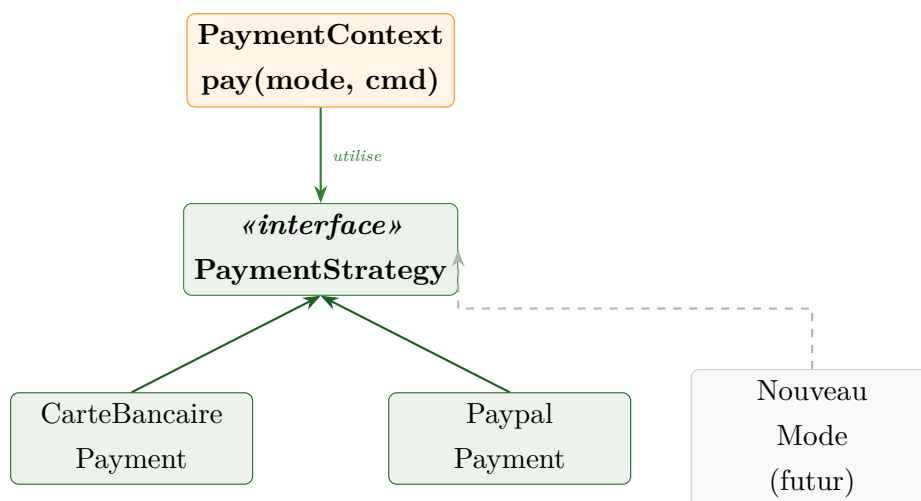


FIGURE 3.1 – Patron Strategy appliqué au module de paiement

3.2.5 Règle du Snapshot

La table `lignes_commande` stocke une *copie* des données au moment de la commande; `LigneCommande` n'a pas de `@ManyToOne` vers `Article` :

```

1  @Entity
2  public class LigneCommande {
3      // FK scalaire -- pas de @ManyToOne vers Article
4      @Column(name = "article_id", nullable = false)

```

```
5     private Long articleId;
6
7     // Copies immuables au moment de la commande
8     private String      nomArticle;
9     private BigDecimal prixUnitaire;
10    private Integer      quantite;
11 }
```

Listing 3.4 – Snapshot dans LigneCommande

3.2.6 Verrouillage Optimiste

Pour prévenir la corruption du stock lors d'accès concurrents :

```
1 @Entity
2 public class Article {
3     @Version
4     private Integer version; // Hibernate leve
5                               OptimisticLockException
6     private Integer stock;   // en cas de modification concurrente
7 }
```

Listing 3.5 – Verrouillage optimiste sur Article

3.3 API REST — Endpoints Implémentés

TABLE 3.1 – Principaux endpoints de l’API REST

Méth.	Route	Accès	Description
POST	/api/auth/register	Public	Inscription
POST	/api/auth/login	Public	Connexion (JWT)
GET	/api/articles	Public	Catalogue
GET	/api/articles/search?q=	Public	Recherche
POST	/api/articles	Admin	Créer un article
PUT	/api/articles/{id}	Admin	Modifier
DELETE	/api/articles/{id}	Admin	Supprimer
GET	/api/bouquet	Client	Voir son bouquet
POST	/api/bouquet/ajouter	Client	Ajouter un article
DELETE	/api/bouquet/{id}	Client	Retirer un article
POST	/api/commandes	Client	Passer une commande
GET	/api/commandes	Client	Lister ses commandes
POST	/api/paiements	Client	Payer
GET	/api/livraisons/commande/{id}	Client	Suivre livraison
GET	/api/admin/commandes	Admin	Toutes les commandes
PATCH	/api/livraisons/{id}/statut	Admin	MAJ statut livraison

3.4 Interface Utilisateur

L’interface frontend est développée en **React 18** avec **Vite** comme outil de build, accessible sur `http://localhost:5173`.

- `src/pages/` : pages principales (accueil, catalogue, bouquet, commandes, paiement, connexion).
- `src/components/` : composants réutilisables (carte article, navigation).
- `src/context/` : Context API pour authentification et bouquet.
- `src/api/` : modules Axios avec intercepteur JWT automatique.
- `src/hooks/` : hooks personnalisés.

CORS : La configuration `SecurityConfig` autorise explicitement l’origine `http://localhost:5173` avec les méthodes GET, POST, PUT, DELETE, OPTIONS et PATCH.

Conclusion et Perspectives

Réalisation des Objectifs

Le projet **Fleur & Atelier** a atteint l'ensemble des objectifs fixés :

- ✓ **Architecture propre** : séparation stricte des couches, dépendances orientées vers le domaine.
- ✓ **API REST complète** : 16 endpoints couvrant l'intégralité du parcours utilisateur et administrateur.
- ✓ **Sécurité JWT** : authentification stateless avec rôles **USER** et **ADMIN**.
- ✓ **Bouquet métier** : entité persistée avec règles métier propres et contrainte d'unicité.
- ✓ **Paiement extensible** : patron Strategy – ajout de modes sans régression.
- ✓ **Atomicité transactionnelle** : paiement + stock + livraison dans une seule transaction `@Transactional`.

Bilan du Projet

Ce projet a permis de mettre en pratique des concepts avancés :

- La **conception UML** en amont a réduit les ambiguïtés architecturales.
- L'application des **principes SOLID** et des design patterns a produit un code maintenable et évolutif.
- La **sécurité par conception** a été intégrée dès les premières décisions d'architecture.

Perspectives d'Évolution

1. **Refresh Token** : renouvellement automatique du JWT sans redéconnexion.
2. **Notifications email** : confirmation de commande via Spring Mail.
3. **Base de données production** : migration vers PostgreSQL avec Flyway pour les migrations.

4. **Tests automatisés** : augmenter la couverture avec JUnit 5, Mockito et Spring Boot Test.
5. **Containerisation** : Docker + Docker Compose pour le déploiement.
6. **Païement réel** : intégration Stripe ou PayPal API.
7. **Stockage cloud** : migration des images vers AWS S3 ou Cloudinary.

Bibliographie

- [1] Pivotal Software, *Spring Boot Reference Documentation*, 2024.
<https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [2] Meta Platforms, Inc., *React – A JavaScript Library for Building User Interfaces*, 2024.
<https://react.dev>
- [3] Evan You, *Vite – Next Generation Frontend Tooling*, 2024.
<https://vitejs.dev>
- [4] Eclipse Foundation, *Jakarta Persistence 3.1 Specification*, 2022.
<https://jakarta.ee/specifications/persistence/3.1/>
- [5] M. Jones, J. Bradley, N. Sakimura, *RFC 7519 – JSON Web Token (JWT)*, IETF, 2015.
<https://www.rfc-editor.org/rfc/rfc7519>
- [6] R. C. Martin, *Clean Architecture : A Craftsman’s Guide to Software Structure and Design*, Prentice Hall, 2017. ISBN 978-0134494166.
- [7] Spring Team, *Spring PetClinic Sample Application*, GitHub, 2024.
<https://github.com/spring-projects/spring-petclinic>